

Case Study: RMF Demos Fleet Adapter

The Python implementation of the **Full Control** fleet adapter classes is demonstrated in the [demos fleet adapter](#). Building on top of the fleet adapter template, the demos fleet adapter uses REST API as an interface between the adapter and the simulation robots: the adapter sends out commands to the robots, while the robots update the adapter on their current state information. This is done by creating a `fleet_manager` node that contains the necessary REST endpoints for `RobotClientAPI` to interact with.



Demos Fleet Manager

Whenever a command is ready to be sent from the RMF fleet adapter, it will call the relevant API function defined in `RobotClientAPI` and query the corresponding endpoint from the API server in the `fleet_manager` node. Each function either retrieves specific information about the robot's current state (including but not limited to its last known position, remaining battery level, and whether it has completed a request), or sends a command to the robot to carry out a request. The robot state information is required for the fleet adapter to update the traffic schedule, plan for subsequent tasks, and guide robots across different paths in the environment.

The demos fleet adapter is integrated with the simulation robots which publish their state information via internal ROS2 messages, hence the `fleet_manager` also serves to consolidate the messages published by different robots from its fleet and sends them to the designated robot's fleet adapter. The API endpoints are designed such that the adapter can query information or send commands to a particular robot by specifying the robot's name. The `fleet_manager` will ensure that the robot exists in the fleet before returning the requested state information or relaying commands to the simulation robot. Additionally, the adapter can retrieve the status of the entire fleet's robots.

Fleet Configuration

There are four **Full Control** fleets in our demo simulation, each with their own fleet-specific parameters. To better consolidate and set up these configurations upon initializing the fleet, they are stored in a `config.yaml` file. Paths to the configuration and navigation graph files are

required when running the fleet adapter and manager. General fleet settings and capabilities are defined under the `rmf_fleet` section.

The config file also takes care of robot-specific parameters within the fleet, such as the number of robots in the fleet, each of their names and their starting waypoints. For example, the `tinyRobot` fleet in Office demo world has two robots, so we append the configurations for each robot to the `robots` section in the config file.

For users who are operating their robots in a different coordinate frame from RMF, the `reference_coordinate` section in the config file helps to perform any necessary transformations. Do note that RMF and the slotcar simulation robots share the same coordinate frame, so this transformation is not implemented in the demos fleet adapter.

Found a bug? [Edit this page on GitHub](#).

Fleet Adapter Tutorial (Python)

`fleet_adapter` acts as a bridge between the robots and the core RMF system.

Its responsibilities include but are not limited to:

- Updating the traffic schedule with the fleet robot's positions
- Responding to tasks
- Controlling the vendor robots.

The `fleet_adapter` receives information (position, current ongoing tasks, battery levels etc.) about each robot in the fleet and sends them to the core RMF system for task planning and scheduling.

- When the core RMF system has a task to dispatch, it communicates with the various fleet adapters to check which fleet is suitable for taking this task.
- It sends a request, to which fleet adapters respond by submitting their fleet robots' availability and statuses.
- RMF determines the best fleet for the task and responds to the winning bid, i.e. the fleet that is selected. The response contains navigation commands relevant to the delegated task.
- The fleet adapter will then send the navigation commands to the robot in appropriate API.

The tutorial provided below is based on the [rmf_demos_fleet_adapter](#) implemented in the [rmf_demos](#) repository. This specific implementation is written in Python and uses REST API as an interface between the fleet adapter and fleet manager. You may choose to use other APIs for your own integration.

1. Pre-requisites

Fetch dependencies

Before running your fleet adapter, make sure that you have ROS 2 and RMF installed by following the instructions [here](#). You have the option of installing the binaries or building from

source for both. You may also wish to head over to our [RMF Github repo](#) for the latest updates and instructions for RMF installation.

If you built ROS 2 and/or RMF from source, make sure to source the workspace that contains their built code before proceeding to the next step.

In our example, the `rmf_demos_fleet_adapter` uses REST API as an interface between the fleet adapter and robot fleet manager, hence to get the demos working we will need to install the required dependencies to use FastAPI.

```
pip3 install fastapi uvicorn
```

This step is only required for this implementation; depending on what API your own fleet manager uses, you'll have to install any necessary dependencies accordingly.

Get started with the fleet adapter template

Create a workspace and clone the [fleet_adapter_template](#) repository.

```
mkdir -p ~/rmf_ws/src  
cd ~/rmf_ws/src/  
git clone https://github.com/open-rmf/fleet_adapter_template.git
```

This template contains the code for both Full Control and Easy Full Control fleet adapters. Both implementations use API calls in [RobotClientAPI.py](#) to communicate with the robots.

2. Update the config.yaml file

The `config.yaml` file contains important parameters for setting up the fleet adapter. Users should start by updating these configurations describing their fleet robots.

It is important to stick to the provided fields in the sample `config.yaml` below, otherwise there will be import errors when parsing this YAML file to the fleet adapter. If you would like to edit any of the field names or value range, or even append additional fields, please ensure that you also modify the part of your fleet adapter code that handles this configuration import accordingly.

Some fields are optional as indicated below.

```

# FLEET CONFIG =====
# RMF Fleet parameters

rmf_fleet:
  name: "tinyRobot"
  limits:
    linear: [0.5, 0.75] # velocity, acceleration
    angular: [0.6, 2.0] # velocity, acceleration
  profile: # Robot profile is modelled as a circle
    footprint: 0.3 # radius in m
    vicinity: 0.5 # radius in m
  reversible: True # whether robots in this fleet can reverse
  battery_system:
    voltage: 12.0 # V
    capacity: 24.0 # Ahr
    charging_current: 5.0 # A
  mechanical_system:
    mass: 20.0 # kg
    moment_of_inertia: 10.0 #kgm^2
    friction_coefficient: 0.22
  ambient_system:
    power: 20.0 # W
  tool_system:
    power: 0.0 # W
  recharge_threshold: 0.10 # Battery level below which robots in this fleet will
not operate
  recharge_soc: 1.0 # Battery level to which robots in this fleet should be
charged up to during recharging tasks
  publish_fleet_state: 10.0 # Publish frequency for fleet state, ensure that it is
same as robot_state_update_frequency
  account_for_battery_drain: True
  task_capabilities: # Specify the types of RMF Tasks that robots in this fleet
are capable of performing
    loop: True
    delivery: True
  actions: ["teleop"]
  finishing_request: "park" # [park, charge, nothing]
  responsive_wait: True # Should responsive wait be on/off for the whole fleet by
default? False if not specified.
  robots:
    tinyRobot1:
      charger: "tinyRobot1_charger"
      responsive_wait: False # Should responsive wait be on/off for this
specific robot? Overrides the fleet-wide setting.
    tinyRobot2:
      charger: "tinyRobot2_charger"
      # No mention of responsive_wait means the fleet-wide setting will be used

  robot_state_update_frequency: 10.0 # Hz

fleet_manager:
  prefix: "http://127.0.0.1:8080"
  user: "some_user"

```

```
password: "some_password"
```

```
# TRANSFORM CONFIG =====
# For computing transforms between Robot and RMF coordinate systems

# Optional
reference_coordinates:
  L1:
    rmf: [[20.33, -3.156],
          [8.908, -2.57],
          [13.02, -3.601],
          [21.93, -4.124]]
    robot: [[59, 399],
            [57, 172],
            [68, 251],
            [75, 429]]
```

- `rmf_fleet` : Important fleet parameters including vehicle traits, task capabilities and user information for connecting to the fleet manager.
 - `limits` : Maximum values for linear and angular accelerations and velocities.
 - `profile` : Radius of the footprint and personal vicinity of the vehicles in this fleet.
 - `reversible` : A flag to enable/disable reverse traversal in the robot.
 - `battery_system` : Information about the battery's voltage, capacity and charging current.
 - `recharge_threshold` : Sets a value for minimum charge below which the robot must return to its charger.
 - `recharge_soc` : The fraction of total battery capacity to which the robot should be charged.
 - `task_capabilities` : The tasks that the robot can perform between `loop`, `delivery` and `clean`.
 - `account_for_battery_drain` : Whether RMF should consider the battery drain of the robots before dispatching tasks.
 - `action` [Optional]: A list of custom performable actions for the fleet.
 - `finishing_request` : What the robot should do when it finishes its task, can be set to `park`, `charge` or `nothing`.

- `responsive_wait` [Optional]: True # Should responsive wait be on/off for the whole fleet by default? False if not specified.
- `robots` : Information about each individual robot in the fleet. Each item in this section corresponds to the configuration for a single robot in the fleet. You may add more robots accordingly.
 - `tinyRobot1` : Name of the robot.
 - `charger` : Name of the robot's charging point.
 - `responsive_wait` : Whether this specific robot should turn its responsive wait on/off. Overrides the fleet-wide setting.
- `robot_state_update_frequency` : How frequently should the robots update the fleet.
- `fleet_manager` : The *prefix*, *user* and *password* fields that can be configured to suit your chosen API. Do make sure to also edit the corresponding fields in `RobotClientAPI.py` if you do modify them. These parameters will be used to set up connection with your fleet manager/robots.
- `reference_coordinates` [Optional]: If the fleet robots are not operating in the same coordinate system as RMF, you can provide two sets of (x, y) coordinates that correspond to the same locations in each system. This helps with estimating coordinate transformations from one frame to another. A minimum of 4 matching waypoints is recommended.

Note: this is not being implemented in `rmf_demos_fleet_adapter` as the demos robots and RMF are using the same coordinate system.

3. Create navigation graphs

A navigation graph is required to be parsed to the fleet adapter so that RMF can understand the robots' environment. They can be created using the [RMF Traffic Editor](#) and the [building_map_generator nav](#) CLI provided. Refer to the traffic editor repo's README for installation and map generation instructions.

You may also want to look through the [Traffic Editor](#) section of this Book for detailed information and instructions on creating your own digital maps.

You should now have a YAML file with information about the lanes and waypoints (among other information) that describe the paths your robot fleet can take.

4. Fill in your RobotAPI

`RobotClientAPI.py` provides a set of methods being used by the fleet adapter. These callbacks are triggered when RMF needs to send or retrieve information via the fleet adapter to/from the managed robots. To cater to the interface of your choice, you need to fill in the missing code blocks marked with `# IMPLEMENT YOUR CODE HERE #` within `RobotAPI` with logics to send or retrieve the corresponding information. For example, if your robot uses REST API to interface with the fleet adapter, you will need to make HTTP request calls to the appropriate endpoints within these functions.

You may refer to the `RobotAPI` class implemented for `rmf_demos_fleet_adapter` for examples of how these methods can be filled up.

- `navigate` : Sends a navigation command to the robot API. It takes in the destination coordinates from RMF, desired map name and optional speed limit.
- `start_activity` : Sends a command to the robot to start performing a task. This method is helpful for custom performable actions that are triggered by `execute_action()` .
- `stop` : Commands the robot to stop moving.
- `position` , `map` and `battery_soc` : Retrieves the robot's current position in its coordinate frame in the format `[x, y, theta]` , its current map name, and its battery state of charge. In `rmf_demos_fleet_adapter` these methods are consolidated under `get_data()` .
- `is_command_completed` : Checks if the robot has completed the ongoing process or task. In `rmf_demos_fleet_adapter` , this is implemented under the `RobotUpdateData` class. Depending on your robot API you may choose to integrate it either way. This callback will help RMF recognize when a dispatched command is completed, and proceed to send subsequent commands.

Further parameters may be added to `RobotAPI` to be used in these callbacks if required, such as authentication details and task IDs. You may also wish to write additional methods in either `RobotAPI` and `fleet_adapter.py` for specific use cases. The `rmf_demos_fleet_adapter` implementation demonstrates this for a `Teleoperation` action, which will be elaborated more in the [PerformAction tutorial](#).

5. Create your fleet adapter!

Now that we have our components ready, we can start creating our fleet adapter.

`fleet_adapter.py` uses the Easy Full Control API to easily create an `Adapter` instance and set up the fleet configurations and robots by parsing the configuration YAML file that we have prepared previously. Since we have defined our `RobotAPI` , the methods implemented will be

used by the callbacks in `fleet_adapter.py` so that RMF can retrieve robot information and send out navigation or action commands appropriately.

You may wish to use the `fleet_adapter.py` available from the fleet adapter template and modify it according to what you'd like your fleet to achieve.

6. Run your fleet adapter

At this point, you should have 4 components ready in order to run your fleet adapter:

- `fleet_adapter.py`
- `RobotClientAPI.py`
- Fleet `config.yaml` file
- Navigation graph

Build your fleet adapter package

If you cloned the `fleet_adapter_template` repository, you would already have your Python scripts in a ROS 2 package. Otherwise, you can follow the instructions [here](#) to create a package in your workspace. For the instructions below, we will use the package and module names used in the `fleet_adapter_template` package.

With your scripts in the appropriate folder, go back to the root directory of your workspace and build the package.

```
colcon build --packages-select fleet_adapter_template
```

Run!

We will now source our workspace and run the fleet adapter:

```
. ~/rmf_ws/install/setup.bash
```

```
ros2 run fleet_adapter_template fleet_adapter -c <path-to-config> -n <path-to-nav-graph>
```

7. Deep dive into the code [Optional]

The following steps elaborate on the Easy Full Control fleet adapter and what each part of the code does.

a. Import important parameters and create an Adapter

When running our fleet adapter, we will need to parse in the fleet config file and navigation graphs that we created in earlier steps. These files will be passed to the EasyFullControl API to set up fleet configurations for the adapter.

```
config_path = args.config_file
nav_graph_path = args.nav_graph

fleet_config = rmf_easy.FleetConfiguration.from_config_files(
    config_path, nav_graph_path
)
assert fleet_config, f'Failed to parse config file [{config_path}]'

# Parse the yaml in Python to get the fleet_manager info
with open(config_path, "r") as f:
    config_yaml = yaml.safe_load(f)
```

With these parameters, we can create an Adapter instance and add an EasyFullControl fleet to it. We would also want to configure the `use_sim_time` and `server_uri` parameters if the adapter should operate according to simulation clock or broadcast task updates to any websocket servers.